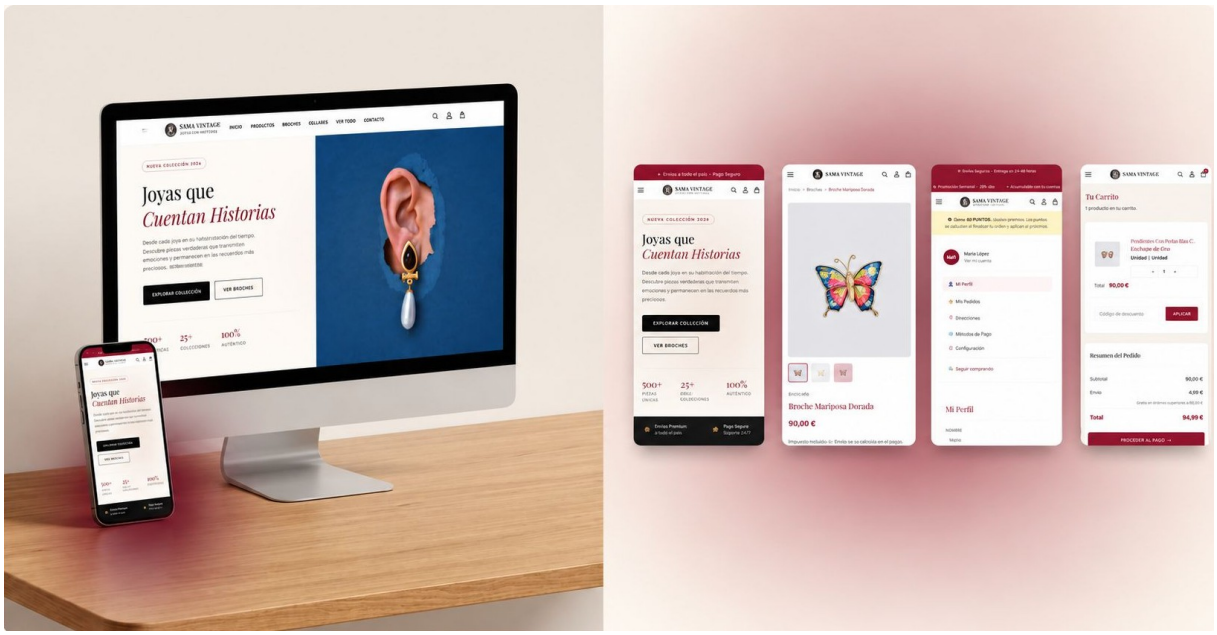


Sama Vintage

Rebranding y tienda online de joyería vintage

Eva Gallardo Romero

13 de marzo de 2026

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **2**

4 Requisitos del sistema **3**
4.1 Requisitos funcionales 3
4.2 Requisitos no funcionales 3

5 Documentación técnica **4**
5.1 data.js: catálogo 4
5.2 data.js: Carrito 4
5.3 templates/tienda.html 6
5.4 templates/producto.html 6
5.5 header.js 7

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: sama-vintage.

La tienda está publicada en evagr11.github.io/sama-vintage, el diseño en Figma, la guía de estilos en Canva y la documentación del código generada con JSDoc, en evagr11.github.io/sama-vintage/docs.

1 Descripción general

Sama Vintage es una tienda real de joyería vintage. En el módulo de **Sistemas de Gestión Empresarial** de DAM había que hacer el **rebranding** completo de una web que ya existiera, y la elegimos: nueva guía de estilos en Canva, prototipos en Figma y la tienda montada desde cero.

- **Fase 1, en equipo**: la primera versión la hicimos Ana Núñez, Ángel Millán y yo en **WordPress** con WooCommerce, un tema personalizado y plugins como Code Snippets, Contact Form 7 y Simple Custom CSS and JS.
- **Fase 2, mía**: después la rehíce yo sola como **web estática** en HTML, CSS y JavaScript, sin CMS ni frameworks, para poder publicarla gratis en GitHub Pages y conservarla en mi portfolio. Esta documentación es de esta versión.

La versión estática tiene un **catálogo de 53 piezas** (broches, pendientes y collares) con filtro por categoría y orden por precio, una **ficha de producto** única que lee el `?id=` de la dirección, un **carrito** guardado en el navegador y un **checkout simulado**, sin pago real. Las fotos se pasaron a **WebP** con carga diferida: el catálogo pasó de 171 MB a menos de 10 MB.

2 Diagrama de clases

Todos los datos salen de `data.js`: el array con las 53 piezas y el objeto `Carrito`. Cada página carga ese archivo y `header.js`, y pinta su contenido con un pequeño script propio.

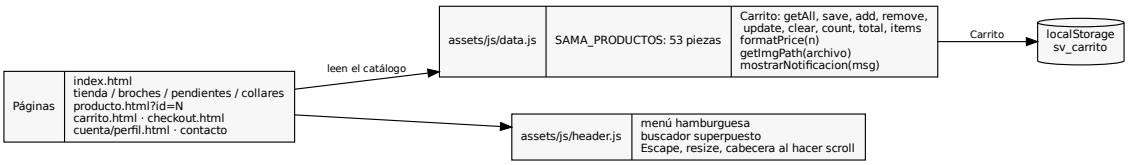
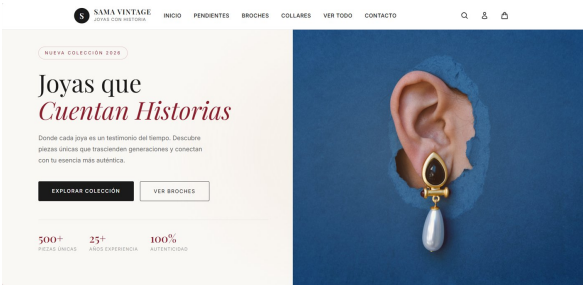


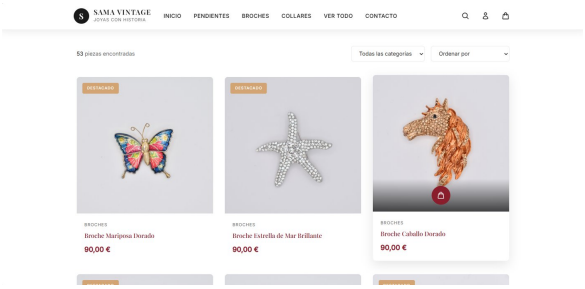
Figura 1: Estructura de la web estática

3 Diseño

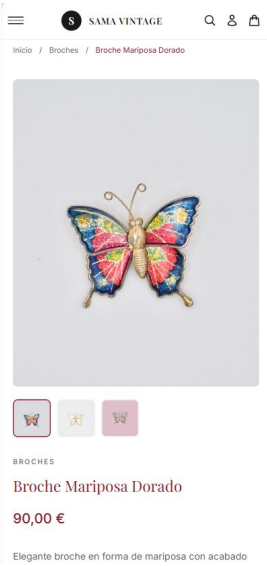
La paleta de la guía de estilos es **granate, dorado y crema**, con títulos en *Playfair Display* para dar el aire clásico de una joyería. La cabecera tiene el logo, el menú, el buscador y el carrito con su contador; en el móvil el menú pasa a un panel que se abre con el botón hamburguesa.



Inicio



Catálogo



Ficha en el móvil



Carrito en el móvil

Figura 2: Capturas de Sama Vintage

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La tienda debe mostrar el **catálogo** con foto, categoría, nombre y precio, y marcar las piezas destacadas.
RF02	El catálogo se debe poder **filtrar** por categoría y por texto, y **ordenar** por precio, nombre o destacados.
RF03	Una única página de **producto** debe mostrar la pieza que indica el `?id=` con su galería y piezas relacionadas.
RF04	Se deben poder **añadir** piezas al carrito, cambiar cantidades y quitarlas.
RF05	El **contador** del carrito debe estar al día en todas las páginas.
RF06	El carrito debe mostrar subtotales y total, y llevar a un **checkout** simulado.
RF07	Debe haber páginas de contacto, cuenta, privacidad y términos.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	Web estática en **HTML, CSS y JavaScript** sin CMS ni frameworks, publicada en **GitHub Pages**.
RNF02	Un único archivo de datos (`data.js`) para todo el catálogo.
RNF03	El carrito se guarda en el **localStorage** del navegador.
RNF04	Imágenes en **WebP** con carga diferida.
RNF05	Diseño **responsive** con menú hamburguesa en móvil.
RNF06	El checkout no cobra nada: es una simulación.
RNF07	El código debe tener comentarios **JSDoc**.

5 Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado.

5.1 data.js: catálogo

5.1.1 Ubicación

`assets/js/data.js`

Descripción general:
El array `SAMA_PRODUCTOS` con las 53 piezas. Es la única fuente de datos de la web.

5.1.2 Funciones

SAMA_PRODUCTOS **Descripción:** Cada pieza tiene id, nombre, precio en euros, categoría, si es destacada, la foto principal, la galería y la descripción.

Código comentado:

```
const SAMA_PRODUCTOS = [
  { id:1, nombre:'Broche Mariposa Dorado', precio:90, categoria:'broches', destacado:
      true,
    img:'BORCHE-MARIPOSA-DORADO-1.webp',
    imagenes:['BORCHE-MARIPOSA-DORADO-1.webp','BORCHE-MARIPOSA-DORADO-2.webp','BORCHE-
        MARIPOSA-DORADO-3.webp'],
    descripcion:'Elegante broche en forma de mariposa con acabado dorado. ...' },
  // ... 53 piezas
];
```

5.2 data.js: Carrito

5.2.1 Ubicación

`assets/js/data.js`

Descripción general:
Objeto con todas las operaciones del carrito. Solo guarda el id y la cantidad de cada pieza; el precio y el nombre se buscan en el catálogo.

5.2.2 Funciones

getAll y save **Descripción:** Lee y guarda la lista en el `localStorage`. Al guardar, actualiza el contador de la cabecera.

Código comentado:

```
_key: 'sv_carrito',
getAll() {
  try { return JSON.parse(localStorage.getItem(this._key)) || []; }
  catch { return []; }                          // si está estropeado, carrito vacío
},
save(items) {
  localStorage.setItem(this._key, JSON.stringify(items));
  this._updateBadge();
},
```

add, update y remove **Descripción:** Añadir suma a la cantidad si la pieza ya estaba; poner una cantidad de 0 o menos la quita.

Código comentado:

```
add(productId, cantidad = 1) {
  const items = this.getAll();
  const idx = items.findIndex(i => i.id === productId);
  if (idx >= 0) items[idx].cantidad += cantidad;
  else items.push({ id: productId, cantidad });
  this.save(items);
},
remove(productId) {
  this.save(this.getAll().filter(i => i.id !== productId));
},
update(productId, cantidad) {
  if (cantidad <= 0) { this.remove(productId); return; }
  const items = this.getAll();
  const idx = items.findIndex(i => i.id === productId);
  if (idx >= 0) { items[idx].cantidad = cantidad; this.save(items); }
},
```

total, items y _updateBadge **Descripción:** Calculan el total y la lista con los datos de cada pieza, y escriben el número de artículos en todos los contadores de la página.

Código comentado:

```
total() {
  return this.getAll().reduce((s, i) => {
    const p = SAMA_PRODUCTOS.find(p => p.id === i.id);
    return s + (p ? p.precio * i.cantidad : 0);
  }, 0);
},
items() {
  return this.getAll().map(i => {
    const p = SAMA_PRODUCTOS.find(p => p.id === i.id);
    return p ? { ...p, cantidad: i.cantidad, subtotal: p.precio * i.cantidad } : null;
  }).filter(Boolean);                           // ignora ids que ya no existen
},
_updateBadge() {
  const c = this.count();
  document.querySelectorAll('.cart-count, #cartCount').forEach(el => {
    el.textContent = c;
    el.classList.toggle('visible', c > 0);
  });
}
```

formatPrice **Descripción:** Formatea un precio con dos decimales, coma y el símbolo del euro.

Código comentado:

```
function formatPrice(n) {
  return parseFloat(n).toFixed(2).replace('.', ',') + ' EUR';
}
```

5.3 templates/tienda.html

5.3.1 Ubicación

`templates/tienda.html (script de la página)`

Descripción general:
El catálogo con filtros y orden.

5.3.2 Funciones

filtrar **Descripción:** Aplica la categoría, el texto buscado y el orden elegidos y vuelve a pintar.

Código comentado:

```
function filtrar(searchQuery) {
  let productos = [...todosProductos];
  const cat   = document.getElementById('filtroCategoria').value;
  const orden = document.getElementById('filtroOrden').value;
  const q = searchQuery !== undefined ? searchQuery
          : (document.getElementById('searchInput')?.value.trim().toLowerCase() || '')
              ;
  if (cat) productos = productos.filter(p => p.categoria === cat);
  if (q)   productos = productos.filter(p => p.nombre.toLowerCase().includes(q));
  if (orden === 'precio-asc')       productos.sort((a,b) => a.precio - b.precio);
  else if (orden === 'precio-desc') productos.sort((a,b) => b.precio - a.precio);
  else if (orden === 'nombre')      productos.sort((a,b) => a.nombre.localeCompare(b.
      nombre,'es'));
  else if (orden === 'destacados')  productos.sort((a,b) => b.destacado - a.destacado)
      ;
  renderProductos(productos);
}
```

5.4 templates/producto.html

5.4.1 Ubicación

`templates/producto.html (script de la página)`

Descripción general:
Una sola plantilla para las 53 fichas.

5.4.2 Funciones

Carga de la ficha **Descripción:** Lee el `id` de la dirección, busca la pieza y, si no existe, muestra un mensaje con enlace a la tienda. Después pinta la ficha y tres piezas de la misma categoría.

Código comentado:

```
const id = parseInt(new URLSearchParams(window.location.search).get('id'));
const p = SAMA_PRODUCTOS.find(x => x.id === id);

if (!p) {
  document.getElementById('productMain').innerHTML = '... Producto no encontrado ...
      Volver a la tienda ...';
  return;
}

document.title = `${p.nombre} - Sama Vintage`;
const relacionados = SAMA_PRODUCTOS.filter(x => x.categoria === p.categoria && x.id
    !== p.id).slice(0, 3);
// ... se pinta la galería, el precio, la descripción y el botón de añadir al carrito
```

5.5 header.js

5.5.1 Ubicación

`assets/js/header.js`

Descripción general:
Comportamiento común de la cabecera en todas las páginas.

5.5.2 Funciones

openMenu y closeMenu **Descripción:** Abren y cierran el menú del móvil bloqueando el scroll de la página mientras está abierto. Se cierra también con `Escape`, al pulsar fuera o al pasar a pantalla de escritorio.

Código comentado:

```
function openMenu() {
  mainNav.classList.add('active');
  menuToggle.classList.add('active');
  menuToggle.setAttribute('aria-expanded', 'true');
  document.body.style.overflow = 'hidden';
  document.body.style.height = '100vh';
}

function closeMenu() {
  mainNav.classList.remove('active');
  menuToggle.classList.remove('active');
  menuToggle.setAttribute('aria-expanded', 'false');
  document.body.style.overflow = '';
  document.body.style.height = '';
}

window.addEventListener('resize', function() {
  if (window.innerWidth > 1024) closeMenu();      // en escritorio no hay menú móvil
});
```